Guia de estudo — `sunnyland_5_fases`

Material didático sobre a versão experimental do ZIP `sunnyland_5_fases`, organizada em uma arquitetura diferente do projeto atual. A ideia é ajudar o grupo a entender o desenho técnico e preparar uma apresentação, não integrar essa cópia automaticamente ao jogo principal.

> *Base da análise: conteúdo do pacote com raiz interna* `sunnyland_5_fases/sunnyland_trabalho_5_fases/`*. A arquitetura e os mapas descritos abaixo são os do projeto compactado, não os mapas em cena do projeto atual. O código foi lido estaticamente; a documentação não afirma que todas as mecânicas foram testadas em jogo.*

1. Resumo em poucas linhas

Esta versão usa geração de fases dirigida por dados. As cinco cenas de nível são pequenas: guardam o id e usam o mesmo `scripts/construtor_fase.gd`. Os desenhos dos mapas estão em texto dentro de `scripts/niveis.gd`; o construtor lê caracteres como `#`, `P`, `g`, `M` e `B` e cria tiles, jogador, inimigos, itens e objetos no jogo.

text

```
fases/2/fase.tscn (idFase = 2)
              │
              ▼
scripts/construtor_fase.gd
        ├── scripts/niveis.gd → dados, tema e mapa textual
        ├── cria TileMap / cenário / colisões
        ├── cria jogador / inimigos / objetos / itens
        └── cria HUD / saída / progressão
```

O nome “procedural” aqui significa que o cenário é montado por código; não significa que o mapa é sorteado a cada partida. Os mapas são desenhados e fixos, armazenados como segmentos de texto.

2. Conceitos do Godot usados pela versão

Recurso	Uso nesta arquitetura
`Node2D` e árvore de nós	Raiz da fase gerada e organização dos objetos criados.
`PackedScene`	Cenas reutilizáveis carregadas como jogador, gambá, sapo, águia, itens e UI.
`TileMap` / `TileSet`	Representa terreno e plataformas e associa colisões às peças.
GDScript	Interpreta dados dos mapas e implementa movimento, HUD, inimigos e objetos.
`Resource`	`dadosFase` representa id, nome, conclusão e desbloqueio.
Autoload	`controladorFases` e `Menu` ficam acessíveis globalmente.
Sinais	Conectam coleta, chaves, dano, chefe, porta e eventos de fase.
`user://`	Guarda progresso local, fora dos arquivos do projeto, em `progresso.cfg`.
`CharacterBody2D`	Movimento físico de jogadores e inimigos terrestres.
`Area2D`	Coleta, chave, checkpoint, porta, espinhos e regiões de perigo.
`CanvasLayer` e controles UI	HUD, pausa, conclusão, mensagens e barra do chefe.
`Tween`, partículas, luzes e parallax	Feedback visual, atmosfera e movimento do cenário.

3. Ciclo completo da geração de uma fase

3.1 Começo do jogo

1.. `project.godot` define `principal.tscn` como cena inicial e configura os

serviços globais.

2.. A cena principal chama `scripts/principal.gd`.

3.. O menu inicial pede a fase id `0`.

4.. O seletor oferece ids `1` a `5`. O controlador consulta progresso e carrega

`res://fases/<id>/fase.tscn`.

3.2 A cena leve escolhe os dados

Cada `fases/N/fase.tscn` anexa `scripts/construtor_fase.gd` e define `idFase = N`. No início do construtor:

1.. `Niveis.obter(idFase)` escolhe um dicionário fixo.

2.. O dicionário informa nome, tema, atributos dos inimigos e parâmetros de

objetos.

3.. Os segmentos de texto do mapa são concatenados e alinhados.

4.. Cada célula do texto é interpretada pela legenda.

5.. O gerador instancia cenas reutilizáveis no local calculado.

6.. O tema escolhe paleta, terreno, fundo, partículas e efeitos.

7.. O sistema cria a interface, as condições de saída e os limites da fase.

3.3 Alinhamento dos segmentos

• A unidade do mapa é uma célula de 16×16 pixels.

• Cada nível contém vários segmentos desenhados como linhas de caracteres.

• O construtor alinha os segmentos pelo chão, lida com diferenças verticais e

forma o mapa completo.

• A escolha das variações de tile é determinística pela posição; não é um

sorteio novo de layout.

• A saída final pode ter até 166 células de largura (aproximadamente 2.656 px,

sem contar outros efeitos/posicionamentos).

4. Legenda do mapa textual

As maiúsculas e minúsculas podem significar coisas diferentes. Em especial, `B` (chefe) não é o mesmo símbolo que `b` (morcego).

Símbolo	Significado
`#`	Terreno sólido.
`=`	Plataforma.
`L`, `H`	Segmentos/tipos de escada.
`P`	Posição inicial do jogador.
`D`	Saída da fase.
`C`	Checkpoint.
`K`	Chave.
`g`, `c`	Gema/diamante e cereja.
`o`, `f`, `e`, `h`, `b`	Tipos de inimigo (gambá, sapo, águia, variante de águia/morcego, conforme a legenda do construtor).
`B`	Chefe.
`^`	Espinhos.
`M`, `V`	Plataformas móveis horizontais e verticais.

Símbolo	Significado
`X`	Plataforma que cai.
`t`	Tocha.
`1–7`	Adereços/decor.
`.`	Espaço vazio.

Para a regra exata de cada caractere, a fonte de verdade é a legenda em `scripts/construtor_fase.gd`, linhas 10–18; os blocos de nível ficam em `scripts/niveis.gd`.

5. Os cinco níveis

As dimensões são contagens de células. Como a célula mede 16 px, uma largura de 154 células equivale a cerca de 2.464 px de percurso horizontal.

Fase	Tema e dimensões	Elementos que ensinam o tema
1 — Floresta Ensolarada	`floresta`, 154×16 células	Uma chave; gambás com velocidade 35, uma vida e patrulha de 48; 13 diamantes e 2 cerejas no mapa. É a amostra básica para entender mapa, coleta e saída.
2 — Pântano ao Entardecer	`pantano`, 152×16 células	Duas chaves, dois checkpoints, seis sapos, um gambá, 11 diamantes, uma cereja e duas plataformas móveis. Gambá: velocidade 50/uma vida. Sapo: velocidade 55, pulo −260, intervalo de 2 s/uma vida. Plataforma: velocidade 35, deslocamento horizontal 64 e espera de 1 s. O tema inclui troncos, vagalumes/partículas e líquido perigoso.
3 — Caverna Sombria	`caverna`, 166×20 células	Três chaves; gambás e sapos com duas vidas; quatro morcegos; espinhos, escadas e tochas. O tema escuro habilita iluminação do jogador.
4 — Ruínas Celestes	`ceu`, 158×18 células	Três chaves; águias e gambás; dez plataformas que caem, duas plataformas móveis horizontais e uma vertical.
5 — Fortaleza Vulcânica	`vulcao`, 166×18 células	Três chaves comuns e uma chave associada ao chefe; gambás, sapos, águias, morcegos, dez espinhos, cinco plataformas que caem e três móveis. Chefe com oito vidas.

Foco de estudo: fase 2

O mapa da fase 2 não é uma alteração isolada de uma cena `.tscn`: é um bloco `FASE_2` em `scripts/niveis.gd` (aproximadamente linhas 143–261). O construtor usa esse bloco para instanciar o cenário e as regras da fase.

- Dois `K` no mapa pedem duas chaves.
- `M` identifica plataformas móveis; a configuração compartilhada define

velocidade, deslocamento e espera.

- Os `f` representam sapos; `o` representa o gambá, segundo a legenda.
- O tema cria um líquido/área de morte: não é somente uma textura decorativa.
- Checkpoints mudam o ponto de retorno da tentativa.
- Os itens são dados do mapa, não instâncias posicionadas manualmente numa

grande cena editada nó por nó.

6. Mecânicas explicadas passo a passo

Chaves e saída trancada

1.. O construtor conta as chaves exigidas pelo mapa (e a chave do chefe, se houver).

2.. Cada `K` cria um objeto de chave.

3.. A coleta emite um sinal e atualiza a contagem no HUD.

4.. A porta fica fechada enquanto faltarem chaves.

5.. Com os requisitos atendidos, o jogador interage com a porta e a fase termina.

Checkpoint e queda/morte

- O checkpoint marca um local seguro durante a tentativa.
- A posição do checkpoint e as chaves da tentativa são mantidas em memória.
- Ao recarregar depois de morrer, a lógica pode preservar o estado da tentativa.
- O líquido de alguns temas e a zona de morte conduzem à rotina de morte.
- O progresso permanente de fases é gravado em `user://progresso.cfg`; isso é diferente das chaves/checkpoint temporários.

Inimigos e chefe

- O construtor escolhe a cena de inimigo conforme o caractere do mapa.
- Em seguida aplica atributos específicos do dicionário do nível (velocidade, vida, salto, patrulha etc.).
- Morcegos reutilizam a cena da águia com ajustes.
- O chefe deriva do opossum, acrescenta investidas/vida de chefe e emite sinais para a barra e para a chave liberada ao ser derrotado.
- A chave do chefe é criada após a derrota; `B` entra no requisito total.

Plataformas móveis e que caem

- `M` e `V` criam plataformas móveis em eixos horizontal/vertical.
- Os parâmetros do nível determinam velocidade, distância e tempo de espera.
- `X` cria uma plataforma que treme, cai após ser pisada e retorna depois do intervalo.
- São mecânicas independentes do tile de chão fixo.

Pausa, HUD e conclusão

`scripts/hud_fase.gd` cria/organiza título, chaves, mensagens, barra do chefe, pausa e tela de conclusão. O construtor também instancia as interfaces de vida e pontuação reaproveitadas do projeto. Assim, a lógica de mapa pode mudar sem redesenhar manualmente toda a interface de cada fase.

7. Inventário dos scripts

Núcleo da fase e progressão

Arquivo	Papel
`scripts/construtor_fase.gd`	Diretor da geração: interpreta dados, constrói tiles/cenário, cria objetos, inimigos, HUD, chefe e conclusão.
`scripts/niveis.gd`	Fonte dos cinco dicionários, parâmetros, temas e mapas textuais; `obter(id)` seleciona a fase.
`scripts/controladorFases.gd`	Carrega/fecha fases, acompanha progresso e tentativa, música e gravação do progresso.
`scripts/dadosFase.gd`	Recurso de dados com id, nome, conclusão e desbloqueio.
`scripts/fase.gd`	Classe-base de fase, com o id usado pelo construtor.
`scripts/principal.gd`	Inicializa nós/dados e liga a cena ao controlador.
`scripts/menu.gd`	Menu inicial e ações de navegar/sair.
`scripts/hud_fase.gd`	HUD de chaves, título, mensagens, chefe, pausa e conclusão.
`scripts/inimigo.gd`	Base/abstração leve para inimigos.

Métodos úteis de `ConstrutorFase`: `_ready`, `_montar_mapa`, `_criar_tiles`, `_criar_objetos`, `_criar_inimigo`, `_criar_chefe`, `_criar_interface`, `_on_chave_coletada` e `_concluir_fase`. Consultar esses métodos em sequência ajuda a explicar como um caractere vira gameplay.

Jogador e inimigos

Arquivo	Papel
`scrips/player.gd`	Movimento, pulo, coyote time, escalada, vida, dano, morte, entrada e animações do jogador.
`actors/player.tscn`	Cena do jogador moderno instanciada pelo construtor.
`scrips/opossum.gd`	Comportamento base do gambá/opossum.
`scrips/frog.gd`	Comportamento do sapo.
`scrips/eagle.gd`	Comportamento da águia; também pode ser adaptado para morcego pelo construtor.
`scripts/objetos/chefe.gd`	Especialização do inimigo com vida de chefe, investida, sinais de vida/derrota.

Objetos de cenário e perigo

Arquivo	Papel
`scripts/objetos/chave.gd`	Área de coleta, sinal da chave, sprite/feedback.
`scripts/objetos/porta_trancada.gd`	Porta fechada/aberta, requisito de chave e interação do jogador.
`scripts/objetos/checkpoint.gd`	Marca o checkpoint ativo.
`scripts/objetos/espinho.gd`	Cria espinhos e causa dano no contato.
`scripts/objetos/zona_morte.gd`	Trata entrada em área letal e comunica morte ao jogador.
`scripts/objetos/plataforma_cai.gd`	Plataforma que avisa/treme, cai e volta.
`prefab/platformaDirecional.tscn`	Cena de plataforma direcional reutilizável (grafia do arquivo no ZIP).
`scrips/area_secreta_01.gd`	Sistema de limite de câmera/região secreta.
`scrips/area_secreta_02.gd`	Efeito de revelar/ocultar polígono.

Interface, itens e prefabs tradicionais

Arquivo	Papel
`prefab/jogador/jogador.gd`	Jogador legado mantido nos prefabs tradicionais.
`prefab/jogador/vida.gd`	HUD antigo de vida.
`prefab/coletaveis/cereja/cereja.gd`	Coletável de cura/feedback.
`prefab/coletaveis/gema/gema.gd`	Coletável de diamante/feedback.
`prefab/controle/chao.gd`	Chão/limite de queda.
`prefab/controle/final.gd`	Finalização legada.
`prefab/controle/mensagens.gd`	Mensagens de vitória/morte.
`prefab/inimigos/gamba/gamba.gd`	Inimigo gambá da família antiga.
`prefab/inimigos/sapo/sapo.gd`	Inimigo sapo da família antiga.
`fases/0/seletor.gd`	Botões e apresentação do seletor.

Arquivos de dados e cenas

- `fases/1/fase.tscn` a `fases/5/fase.tscn`: cenas pequenas com o construtor e `idFase`.
- `fases/0/fase.tscn`: seletor de níveis.
- `scripts/niveis.gd`: cinco desenhos/definições de fase.
- `assets/environment/world/tiles/terrain.png`: atlas do terreno.
- `assets/environment/background/`: imagens de fundo/camadas de cenário.
- `assets/player/foxy-sprites/`: sprites do personagem.
- `assets/characters/opossum/`, `frog/`, `eagle/`: arte de inimigos.
- `assets/misc/novos/`: inclui chave; o ZIP traz imagens auxiliares de bandeira, cadeado e chave que não aparecem na pasta de assets da cópia atual.
- `assets/musica/walkyandtalky.ogg`: música referenciada pelo projeto.
- `assets/`: ainda contém imagens-fonte e outros recursos necessários ao jogo.

8. Diferença para o projeto atual

Tema	`sunnyland_5_fases`	Projeto atual
Como o nível é montado	Um construtor interpreta mapas textuais em `scripts/niveis.gd`.	Fases principalmente serializadas como cenas/TileMaps editados no Godot.
Onde mora a mudança de layout	Dados de segmento e legenda.	Dados de tiles e nós diretamente em `fases/N/fase.tscn`.
Reuso	Um gerador comum cria as cinco fases com temas/objetos.	Reuso por cenas e scripts; os níveis combinam recursos de gerações diferentes.
Mecânicas extras	Porta/chaves, checkpoints, chefe, espinhos e plataformas móveis/que caem estão ligadas ao gerador.	Há escadas, saída, itens, áreas secretas e outras mecânicas, mas não necessariamente os sistemas de chave/checkpoint/chefe do ZIP.
Assets	Além da base, há recursos de chave, cadeado e bandeira.	Os arquivos de assets comparados da base eram idênticos, mas esses três recursos extras não estavam no projeto atual.
Integração direta	Não é um patch isolado da fase 2: depende do construtor, mapa textual, legenda, recursos e fluxo de progressão.	É a base mais apropriada para manter a integração incremental do grupo.

Portanto, o ZIP é mais útil como referência de arquitetura de dados e ideias de gameplay do que como fonte para copiar arquivos sem adaptação. Este guia é para estudar essa versão; não altera nem substitui o projeto atual.

9. Leitura guiada dos códigos

Esta seção organiza a leitura em ordem de execução e destaca como os arquivos se comunicam. Os números de linha abaixo são referências aproximadas para a versão do ZIP analisada.

9.1 Núcleo: `scripts/construtor_fase.gd`

É o arquivo mais importante da geração. Como é grande (cerca de 840 linhas), estude-o em blocos, em vez de tentar memorizar o arquivo inteiro.

1.. Linhas 1–18 — legenda: define como caracteres do mapa viram terreno,

plataformas, escadas, jogador, saída, inimigos e itens.

2.. Linhas 20–88 — constantes e tema: define tamanho da célula, cenas

pré-carregadas, sprites, blocos de tiles, paletas e recursos visuais.

3.. Linha 128 em diante — `_ready()`: busca os dados pelo `idFase` e

coordena a sequência de criação.

4.. `_montar_mapa()` (por volta da linha 169): alinha e combina os segmentos

que vieram do dicionário.

5.. `_criar_tiles()` (por volta da linha 215): percorre o texto, cria os

tiles e calcula posição/variação.

6.. Linhas 248–514: constroem cenário e ambiente, como fundos, limites,

líquidos/partículas, iluminação e decoração do tema.

7.. `_criar_objetos()` (por volta da linha 515): interpreta símbolos de

jogador, chaves, porta, itens, checkpoint, escadas e perigos.

8.. Linhas 693–750: escolhem o inimigo apropriado e aplicam os parâmetros

da fase; morcegos reaproveitam a cena da águia com ajustes.

9.. `_criar_chefe()` (por volta da linha 752): cria o chefe e liga sinais de

vida/derrota.

10.. `_criar_interface()` (por volta da linha 792): cria HUD de fase, vida e

pontuação.

11.. **`_on_chave_coletada()` e `_concluir_fase()` (por volta das linhas 812 e

832):** atualizam requisitos e finalizam o nível.

Como explicar em sala: o construtor é um tradutor entre o dicionário/mapa textual e a árvore de nós que o Godot executa.

9.2 Dados: `scripts/niveis.gd`

- `obter(id)` faz um `match` e entrega o dicionário de uma das cinco fases.
- Cada bloco (`FASE_1`…`FASE_5`) tem nome, tema, atributos de inimigos,

parâmetros globais de plataforma e um ou mais segmentos de mapa.

- A forma de editar dificuldade sem duplicar o construtor é ajustar os dados

do inimigo no bloco da fase.

- A forma de alterar a rota é editar o texto do segmento, respeitando a largura

consistente das linhas e a legenda de caracteres.

• As dimensões informadas na tabela correspondem a células; não a unidades

arbitrárias nem a pixels.

9.3 Navegação e persistência

`scripts/principal.gd` e `scripts/menu.gd`

• A cena principal disponibiliza o nó que recebe o nível e os dados de fase.

• O menu inicial encaminha para id `0` (seletor) e oferece saída do jogo.

`scripts/controladorFases.gd`

• `carregarFase(id)` carrega a cena leve `fases/id/fase.tscn`.

• A cena instanciada executa o construtor com aquele id.

• Funções próprias mantêm fase atual, liberam próxima fase e registram sucesso.

• `user://progresso.cfg` guarda a progressão permanente do usuário.

• As chaves e o ponto de checkpoint da tentativa não são o mesmo dado que o

progresso permanente; a lógica de tentativa é mantida na memória/recarga.

• O seletor avalia estado e inicia somente níveis liberados, segundo o fluxo

analisado.

`scripts/dadosFase.gd` e `scripts/fase.gd`

Guardam a pequena estrutura de dados da fase: id, nome, conclusão e desbloqueio. As cenas leves indicam o id; o construtor usa o mesmo valor para escolher `Niveis.obter(id)`.

9.4 Movimento e dano do jogador

`scrips/player.gd` controla o jogador criado pelo construtor:

• propriedades exportadas/constantes definem velocidade, pulo, velocidade de

escalada e vida;

• métodos de entrada mapeiam ações horizontais, verticais, pulo e interação;

• o ciclo físico escolhe estado/animação, aplica gravidade e move o corpo;

• a área de escada conversa com os métodos de entrada/saída do estado `CLIMB`;

• a hurtbox recebe dano e inicia ferimento/morte conforme a vida restante;

• ataques de queda sobre inimigos podem usar hitbox, rebote e dano;

• reset e checkpoint posicionam o personagem para a tentativa seguinte.

Parâmetros de referência encontrados no código: velocidade padrão 150, velocidade de escalada 85, vida 3 e coyote time 0,075 s. A configuração de `project.godot` inclui ações `pulo`, `irEsquerda`, `irDireita`, `cima` e `baixo`; o script reconhece também aliases em inglês.

9.5 Inimigos e especialização de chefe

• `scrips/opossum.gd`, `scrips/frog.gd` e `scrips/eagle.gd` implementam

comportamentos-base, animações e resposta a dano.

• O construtor passa valores da fase (por exemplo velocidade, patrulha, vida,

salto e intervalo) em vez de criar um script novo para cada nível.

• A cena do inimigo fornece sprites, colisões, áreas de ataque e hurtboxes.

• `scripts/objetos/chefe.gd` especializa o opossum e acrescenta investida,

preparação, vida de chefe e sinal de derrota.

• Quando o chefe é vencido, o gerador cria/libera a chave adicional da fase 5.

9.6 Objetos: entrada, eventos e resultado

Arquivo	Sequência do comportamento
`scripts/objetos/chave.gd`	Detecta corpo → valida jogador → sinaliza chave coletada → anima feedback/remove objeto.
`scripts/objetos/porta_trancada.gd`	Detecta proximidade → mostra aviso → recebe interação → verifica requisito → troca sprite/abre → sinaliza conclusão.
`scripts/objetos/checkpoint.gd`	Detecta jogador → marca checkpoint ativo → comunica a posição segura.
`scripts/objetos/espinho.gd`	Cria sprite e forma de colisão → detecta jogador → aplica dano.
`scripts/objetos/zona_morte.gd`	Detecta corpo em região letal → chama a rotina de morte do jogador.
`scripts/objetos/plataforma_cai.gd`	Sensor detecta pisada → tremor/aviso → queda → tempo de espera → volta à posição inicial.
`scripts/hud_fase.gd`	Exibe título/chaves/mensagens → atualiza barra do chefe → coordena pausa e tela de conclusão.

`scripts/construtor_fase.gd` liga os objetos e sinais; os scripts individuais implementam a resposta local. Ao analisar uma mecânica, procure tanto o lugar onde ela nasce no mapa quanto o callback que efetua a ação.

9.7 Ferramentas e arquivos auxiliares

• `tools/teste.gd` / `tools/teste.tscn` e

`tools/teste_fisica.gd` / `tools/teste_fisica.tscn` são cenas/scripts auxiliares de teste; não são fases normais do jogador.

• `Fase05.tscn` existe além das cenas do diretório `fases/`; confira sempre o

caminho usado pelo controlador antes de decidir qual cena corresponde ao fluxo de execução.

• Arquivos `.import` e o cache `.godot` são auxiliares do editor/recursos.

Eles não substituem o arquivo fonte de imagem nem os scripts.

• Diretórios `scrips` e `scripts`, e `prefab` e `prefabs`, têm grafias

diferentes; caminhos do Godot precisam coincidir exatamente.

10. Limitações e cuidados de leitura

• A lista de símbolos é central: trocar a maiúscula/minúscula altera o objeto

que o construtor cria.

• O construtor é um script grande e concentra responsabilidades; ao depurar,

siga um método por vez, começando por `_ready()` e pelo mapa.

• Existem diretórios com nomes diferentes, como `scrips` e `scripts`, e

`prefab` e `prefabs`. São caminhos separados, não erros de ortografia intercambiáveis.

• O ZIP inclui `Fase05.tscn` e cenas de teste. O controlador usa

`fases/<id>/fase.tscn`; um arquivo extra não significa que esteja no fluxo principal.

• Há scripts próprios de áreas secretas, mas a análise estática não encontrou

ligação deles com os mapas gerados das fases 1–5.

• Recursos do editor/cache podem estar no ZIP; diferencie o que é fonte do que

é importação/cache.

• Uma inspeção estática explica a estrutura, não prova balanceamento,

acessibilidade de todos os itens ou ausência de travamentos no jogo.

11. Plano curto para a apresentação

1.. Mostrar a cena principal e os autoloads.
2.. Abrir uma cena leve, como `fases/2/fase.tscn`, e apontar `idFase = 2`.
3.. Mostrar o bloco `FASE_2` em `scripts/niveis.gd`.
4.. Seguir um símbolo (`M` ou `K`) para a função do construtor que o instancia.
5.. Mostrar a diferença entre cenário decorativo e zona de morte, ou entre mapa fixo e geração aleatória.
6.. Explicar uma mecânica completa: jogador coleta chave → sinal incrementa HUD → porta libera saída.
7.. Comparar esse desenho de dados com a cena desenhada manualmente no projeto atual e concluir por que copiar apenas a fase 2 não é suficiente.

Perguntas para praticar

- Qual é a diferença entre mapa procedural e mapa aleatório?
- Quantas células tem cada tile? Como converter uma largura do mapa para pixels?
- Por que as cenas 1–5 são pequenas?
- O que `Niveis.obter(id)` devolve?
- Como um `K` vira uma chave no mundo?
- O que a chave do chefe muda na fase 5?
- Quais símbolos representam plataformas móveis e plataformas que caem?
- Onde fica salvo o progresso permanente e onde ficam os dados da tentativa?
- Por que o construtor e seus recursos são dependências para portar a fase 2?

12. Arquivos para abrir primeiro

Como a base é um arquivo ZIP, estes caminhos são relativos à raiz interna do pacote:

1.. `project.godot`
2.. `principal.tscn`
3.. `scripts/principal.gd`
4.. `scripts/controladorFases.gd`
5.. `fases/2/fase.tscn`
6.. `scripts/niveis.gd` (bloco `FASE_2`)
7.. `scripts/construtor_fase.gd` (legenda e métodos de montagem)
8.. `scrips/player.gd`
9.. `scripts/hud_fase.gd`
10.. `scripts/objetos/porta_trancada.gd`, `chave.gd` e `checkpoint.gd`